
PREDICTING STOCK PRICES USING PERMUTATION DECISION TREES AND STRATEGIC TRAILING

Vishrut Ramraj

Department of Computer Science and Information Systems
BITS Pilani K K Birla Goa Campus
Goa, 403726, India
vishrut172@gmail.com

Nithin Nagaraj

Complex Systems Programme
National Institute of Advanced Studies, IISc Campus
Bengaluru, 560012, Karnataka, India
nithin@nias.res.in

Harikrishnan N B

Department of Computer Science and Information Systems
BITS Pilani K K Birla Goa Campus
Goa, 403726, India
Adjunct Faculty, Consciousness Studies Programme
National Institute of Advanced Studies, IISc Campus
Bengaluru, 560012, Karnataka, India
harikrishnannb@goa.bits-pilani.ac.in

ABSTRACT

In this paper, we explore the application of Permutation Decision Trees (PDT) and strategic trailing for predicting stock market movements and executing profitable trades in the Indian stock market. We focus on high-frequency data using 5-minute candlesticks for the top 50 stocks listed in the NIFTY 50 index and Forex pairs such as XAUUSD and EURUSD. We implement a trading strategy that aims to buy stocks at lower prices and sell them at higher prices, capitalizing on short-term market fluctuations. Due to regulatory constraints in India, short selling is not considered in our strategy. The model incorporates various technical indicators and employs hyperparameters such as the trailing stop-loss value and support thresholds to manage risk effectively. We trained and tested data on a 3 month dataset provided by Yahoo Finance. Our bot based on Permutation Decision Tree achieved a profit of 1.1802% over the testing period, where as a bot based on LSTM gave a return of 0.557% over the testing period and a bot based on RNN gave a return of 0.5896% over the testing period. All of the bots outperform the buy-and-hold strategy, which resulted in a loss of 2.29%.

Keywords Permutation Decision Tree · Stock Market Trading · Long Short Term Memory · Recurrent Neural Network

1 Introduction

Stock market trading has garnered enormous interest due to its potential for generating revenue. Successful trading combines both intuition, developed through studying market dynamics, and an algorithmic approach that leverages data-driven decision-making. The algorithmic approach involves extracting meaningful features from stock market data—typically time series data—and using this information to decide whether to buy or sell a stock. Advancements in Machine Learning and Deep Learning have significantly expanded the applicability of learning algorithms in stock

market price prediction and classification, aiding traders in making informed decisions. Several studies have focused on predicting NIFTY 50 stock prices using time series models. Fathali et al. (2022) [1] compared the performance of Recurrent Neural Networks (RNN), Long Short-Term Memory (LSTM), and Convolutional Neural Networks (CNN) for this task. Their findings indicated that an LSTM model utilizing High/Low/Open/Close (HLOC) features and a ReLU activation function achieved the best results, reporting a Root Mean Squared Error (RMSE) of 0.002 and an R-squared (R^2) value of 0.537 on the testing dataset. Other research has explored hybrid approaches. Manjunath et al. (2023) [2] proposed a Principal Component Analysis (PCA) combined with various ML models, including Artificial Neural Networks (ANN), Support Vector Machines (SVM) with a Radial Basis Function (RBF) kernel, Naive Bayes, and Random Forest, to predict NIFTY 50 trends. Their hybrid PCA-ANN model achieved a remarkable accuracy and F1-score of 0.9984, while PCA combined with SVM (RBF) and Random Forest yielded an Area Under the Curve (AUC) of 1. Traditional neural network architectures have also been investigated. Kumar et al. (2019) [3] employed a Multi-Layer Perceptron (MLP) with backpropagation to forecast the next day's Open (O), High (H), Low (L), and Close (C) data for the NIFTY 50, reporting an average accuracy of 99.21% with an RMSE of 0.0079. Beyond standard neural networks, more sophisticated techniques like the Adaptive Neuro-Fuzzy Inference System (ANFIS) have been applied. Shankar et al. (2020) [4] demonstrated the effectiveness of ANFIS in modeling risk-return patterns and generating accurate predictions for selected NIFTY 50 blue-chip companies based on adjusted close price, volume, and net profit. In contrast, Singh et al. (2022) [5] evaluated a broader range of eight supervised learning models, including ANN, SVM, Stochastic Gradient Descent (SGD), Linear Regression (LR), AdaBoost, Random Forest (RF), k-Nearest Neighbors (kNN), and Decision Trees (DT), on NIFTY 50 data. Their analysis concluded that Linear Regression and ANN exhibited promising results, while Decision Trees showed signs of overfitting and poor generalization. Volatility forecasting has also been a subject of study. Mahajan et al. (2022) [6] compared symmetric (GARCH) and asymmetric (EGARCH, TAR) Generalized Autoregressive Conditional Heteroskedasticity models with an LSTM model for predicting NIFTY 50 volatility using the India VIX. Their findings indicated that the EGARCH model achieved the best forecasting accuracy, with a Mean Absolute Percentage Error (MAPE) of 16.99% and an RMSE of 5.05, while the LSTM model underperformed across all metrics. Further exploration of ANN architectures includes the work by Lamba et al. (2021) [7], who compared Feedforward Neural Networks (FFNN), Generalized Regression Neural Networks (GRNN), Radial Basis Neural Networks (RBNN), and Enhanced Radial Basis Neural Networks (ERBNN) for predicting NIFTY 50 closing prices. The ERBNN model achieved the highest accuracy of 99.31%, followed by RBNN at 98.17%. Studies comparing traditional ML models have also been conducted. Eswar et al. (2023) [8] compared SVM and Random Forest for NIFTY 50 price prediction using historical data, with Random Forest demonstrating superior performance, achieving an RMSE of 9.75 and an R^2 of 99.2%. Similarly, Sen et al. (2021) [9] compared eight ML models and four LSTM variants, identifying a univariate LSTM model utilizing one week of prior data as the most accurate, with an RMSE/mean ratio of 0.0311. Deep learning models, particularly LSTMs, have gained significant attention. A study by [10] applied an LSTM-based deep learning model to predict the prices of 10 NIFTY 50 stocks, achieving a peak accuracy of 83.88% for State Bank of India (SBI) and an average accuracy of approximately 83% across all analyzed stocks. Furthermore, Jafar et al. (2023) [11] proposed a Backward Elimination-enhanced LSTM (BE-LSTM) model for NIFTY 50 price prediction, reporting superior performance compared to a standard LSTM with 95% accuracy, 3.54% MAPE, and an RMSE of 619.35 after 50 training epochs. Traditional statistical models have also been revisited in conjunction with ML. Patel et al. (2015) [12] compared ANN, SVM, Random Forest, and Naive Bayes models for predicting NIFTY 50 and other indices using trend deterministic data. Their analysis revealed that the Naive Bayes (Multivariate Bernoulli) model achieved the best accuracy of 90.19%, closely followed by Random Forest at 89.98%. Jain et al. (2018) [13] utilized an Artificial Neural Network with backpropagation to predict individual stock prices within the NIFTY 50 financial sector. The study reported moderate accuracy, with error percentages ranging from 6.13% to 16.13% across the different stocks analyzed.

In summary, the reviewed literature demonstrates a significant interest in applying various machine learning models to the NIFTY 50 stock market for prediction and analysis. Despite the extensive application of machine learning (ML) techniques in stock market prediction, most existing methods struggle to achieve robust and consistent returns over short time horizons. Deep learning models such as Long Short-Term Memory (LSTM) networks and Recurrent Neural Networks (RNNs), while adept at capturing sequential dependencies, face several challenges

- **Overfitting:** High model complexity requires vast amounts of data; intraday trading data can be noisy and insufficient to train deep networks effectively without overfitting.
- **Black-Box Interpretability:** Many high-performing ML models (e.g., deep neural networks) lack transparency in their decision-making, making it difficult for traders to trust or refine the outputs.
- **Inconsistent Short-Term Returns:** Volatile market movements lead to unpredictable performance when existing ML models are used for intraday (5-minute) timeframes. Gains from certain periods are often offset by losses in others.

This difficulty arises due to inherent nonlinearity, abrupt fluctuations, and the influence of multiple interdependent factors driving stock market dynamics. In this paper, we focus on high-frequency data using 5-minute candlesticks for the 50 stocks listed in the NIFTY 50 index.

1.1 Problem Definition

The central challenge in algorithmic trading lies in developing a system that not only demonstrates reliability and high performance but also offers interpretability, particularly when utilizing high-frequency data. Traditional approaches often struggle to consistently outperform naive strategies, such as a buy-and-hold approach. This necessitates the development of a robust trading system capable of exploiting short-term (5-minute) price movements.

To address these challenges, this study proposes the application of the *Permutation Decision Tree* (PDT) algorithm by Harikrishnan et al. [14]. PDT, a decision-tree framework driven by the Effort to Compress (ETC) [16] metric, is hypothesized to effectively model patterns in high-frequency stock data. The core hypothesis is that, compared to black-box deep learning models, *PDT can mitigate overfitting risks and offer enhanced interpretability*, thereby improving *risk-adjusted returns* for short-term trading in the Indian stock market.

2 Dataset Description and Feature Engineering

For this study, we utilized intraday 5-minute candlestick data for the NIFTY 50 index's underlying constituent stocks (the list of stocks is available in the Appendix Section 7), a major Forex currency–currency pair (EURUSD), and a major Forex commodity–currency pair (XAUUSD). For NIFTY 50 index's underlying stocks, the data was sourced from Yahoo Finance¹ dated from 2024-10-18, 09:15:00 IST to 2025-01-14, 15:25:00 IST. For the Forex pairs, both XAUUSD and EURUSD, the data was sourced from Stooq² dated from 2025-03-17 to 2025-08-11.

Dataset Overview (NIFTY 50 index's underlying stocks): Each stock has 4,426 data points, resulting in a total of 221,300 data points across all 50 stocks.

Dataset Overview (Forex pairs, XAUUSD and EURUSD): EURUSD has 29,711 data points and XAUUSD has 28,631 data points.

We have considered raw features and features derived from the raw features. They are as follows:

- **Raw Features**

1. **Time:** The timestamp for the data point.
2. **Open (O):** The opening price of the stock during the 5-minute interval.
3. **High (H):** The highest price of the stock during the 5-minute interval.
4. **Low (L):** The lowest price of the stock during the 5-minute interval.
5. **Close (C):** The closing price of the stock during the 5-minute interval.

- **Derived Features**

1. **Swing High:** The most recent swing high price.
2. **Swing Low:** The most recent swing low price.
 - Swing highs and swing lows are critical concepts in technical analysis that help traders identify key turning points in the market. These points represent areas of psychological significance, often acting as resistance (for swing highs) or support (for swing lows).

Definitions:

- * **Swing High:** A price point that is higher than the prices immediately preceding and following it. Swing highs indicate a local peak in the price chart.
- * **Swing Low:** A price point that is lower than the prices immediately preceding and following it. Swing lows indicate a local trough in the price chart.

¹<https://finance.yahoo.com>

²<https://stooq.com/db/h/>

Importance of Swing Highs and Lows:

- * Swing highs often act as **resistance levels**, where prices struggle to rise above.
- * Swing lows often act as **support levels**, where prices struggle to fall below.
- * These points are used to identify trends and reversals, helping traders make informed decisions.

Swing highs and lows are identified by comparing a price point with its neighbors:

- * A price point is a swing high if:

$$P_i > P_{i-1} \quad \text{and} \quad P_i > P_{i+1}$$

- * A price point is a swing low if:

$$P_i < P_{i-1} \quad \text{and} \quad P_i < P_{i+1}$$

3. **Distance to Swing High:** The difference between the current closing price and the swing high price.
4. **Distance to Swing Low:** The difference between the current closing price and the swing low price.
5. **Order Block Indicator:** Binary value indicating the presence of an order block.
 - An **order block** is a price zone where large institutional investors (such as mutual funds, hedge funds, or banks) have placed significant buy or sell orders. These zones often act as strong levels of **support** or **resistance** because of the high volume of trades executed by these bigger players.

Characteristics of Order Blocks:

- * **Consolidation Zone:** Before a large price movement, the stock price often consolidates within a narrow range. This is where large investors are accumulating (buying) or distributing (selling) their positions.
- * **Breakout:** After the accumulation or distribution is complete, the stock breaks out of the range:
 - If the breakout is **upward**, it indicates buying activity by large investors.
 - If the breakout is **downward**, it indicates selling activity by large investors.
- * **Future Price Action:** After a breakout, the order block acts as:
 - A **support level** if the price retraces to the order block after an upward breakout.
 - A **resistance level** if the price retraces to the order block after a downward breakout.

Support and Resistance:

- **Support:** A price level where buying pressure is strong enough to prevent the price from falling further. It acts as a “floor” for the price.
- **Resistance:** A price level where selling pressure is strong enough to prevent the price from rising further. It acts as a “ceiling” for the price.

How to Identify Order Blocks: Order blocks are identified by finding periods of low price volatility followed by high price volatility. In this study, the order block is computed using:

- (a) Calculating the **rolling high** (maximum price) and **rolling low** (minimum price) over a fixed window (e.g., 5 candlesticks).
 - (b) Checking whether the range between the rolling high and low is smaller than a percentage (e.g., 0.2%) of the current closing price.
6. **20-Period Moving Average (MA_20):** The simple moving average of the closing prices over the last 20 intervals.
 7. **50-Period Moving Average (MA_50):** The simple moving average of the closing prices over the last 50 intervals.
 8. **Diff:** The simple difference between MA_20 and MA_50.
 9. **Future Close:** This is the closing price from a variable time steps into the future for each current row.
 10. **Relative Strength Index (RSI):** Momentum oscillator measuring the speed and change of price movements.
 - The **Relative Strength Index (RSI)** is a momentum indicator that measures the speed and magnitude of price changes to determine whether a stock is **overbought** or **oversold**. RSI values range from 0 to 100:
 - * **Overbought** ($RSI > 70$): The stock price has risen too much and might fall soon.
 - * **Oversold** ($RSI < 30$): The stock price has fallen too much and might rise soon.

Formula for RSI: The RSI is calculated using the following formula:

$$RSI = 100 - \left(\frac{100}{1 + RS} \right)$$

where RS (Relative Strength) is defined as:

$$RS = \frac{\text{Average Gain over } n \text{ periods}}{\text{Average Loss over } n \text{ periods}}$$

Steps to Calculate RSI:

- Compute the price changes for each interval ($\Delta P = P_{\text{current}} - P_{\text{previous}}$).
 - Separate the gains ($\Delta P > 0$) and losses ($\Delta P < 0$).
 - Calculate the average gain and average loss over n periods (usually $n = 14$).
 - Compute $RS = \frac{\text{Average Gain}}{\text{Average Loss}}$.
 - Use the RS to calculate RSI.
- Label:** Binary value indicating whether the stock price is expected to increase (1) or decrease (0) over the next 50 intervals (methodology of calculating the target is mentioned in Section 3.2.).

Table 1 and Table 2 provides a sample dataset for Reliance Industries, highlighting the features used in our experiments. The raw data was obtained from Yahoo Finance, containing the essential price information for each 5-minute interval. Table 1 provides a sample of this raw data for Reliance Industries. From the raw data, we calculated several *derived features* which includes technical indicators and features to enhance the predictive power of the model. Table 2 provides a sample of the derived features.

Table 1: Raw Data for Reliance Industries (5-minute Candlestick Data)

Time	Open	High	Low	Close
2024-10-18 13:25	1364.65	1364.90	1363.07	1364.53
2024-10-18 13:30	1364.72	1366.80	1363.70	1366.00
2024-10-18 13:35	1366.20	1366.50	1365.28	1365.82
2024-10-18 13:40	1365.75	1365.75	1364.53	1364.78
2024-10-18 13:45	1364.78	1366.55	1364.53	1365.25
2024-10-18 13:50	1365.25	1365.25	1362.50	1363.95
2024-10-18 13:55	1363.75	1364.80	1363.50	1364.65
2024-10-18 14:00	1364.65	1365.00	1363.95	1364.35
2024-10-18 14:05	1364.20	1365.45	1363.55	1365.25
2024-10-18 14:10	1365.03	1365.90	1364.57	1365.50

Table 2: Derived Features for Reliance Industries

Time	SH	SL	Dist_SH	Dist_SL	OB	MA_20	MA_50	diff	RSI
2024-10-18 13:25	1365.50	1362.50	-0.97	2.03	0	1362.73	1356.42	6.31	57.27
2024-10-18 13:30	1366.80	1362.50	-0.80	3.50	0	1363.06	1356.80	6.26	56.79
2024-10-18 13:35	1366.80	1362.50	-0.98	3.32	0	1363.38	1357.28	6.10	60.97
2024-10-18 13:40	1366.80	1362.50	-2.03	2.28	0	1363.69	1357.74	5.95	60.22
2024-10-18 13:45	1366.80	1362.50	-1.55	2.75	0	1363.94	1358.11	5.82	55.82
2024-10-18 13:50	1366.80	1362.50	-2.85	1.45	0	1364.18	1358.48	5.70	57.34
2024-10-18 13:55	1366.80	1362.50	-2.15	2.15	0	1364.36	1358.83	5.54	52.06
2024-10-18 14:00	1366.80	1362.50	-2.45	1.85	0	1364.58	1359.14	5.44	54.49
2024-10-18 14:05	1365.45	1362.50	-0.20	2.75	0	1364.69	1359.54	5.15	53.24
2024-10-18 14:10	1365.90	1362.50	-0.40	3.00	0	1364.62	1359.91	4.71	56.46

Abbreviations Used: The following abbreviations are used in the dataset to denote various features:

Table 3: Abbreviations and Their Descriptions

Abbreviation	Description
SL	Swing Low: The most recent significant low price.
SH	Swing High: The most recent significant high price.
Dist_SL	Distance to Swing Low: Difference between current close price and Swing Low.
Dist_SH	Distance to Swing High: Difference between current close price and Swing High.
OB	Order Block Indicator: Binary value indicating presence of order block.
MA_20	20-Period Moving Average.
MA_50	50-Period Moving Average.
RSI	Relative Strength Index: Momentum oscillator for price movement.

2.1 Preprocessing

Prior to feature engineering and model training, the raw data underwent a series of preprocessing steps. These procedures ensured data cleanliness, consistency, and alignment across all stocks.

1. Feature Filtering: From the raw data, only the necessary features were retained for further analysis. These include only Close prices
2. Handling Missing Data: In our dataset, we have never encountered missing data, thus ensuring consistency across analysis.
3. Timeframe Consistency: To ensure uniformity, all stock data points were aligned to the same 5-minute intervals.
4. After feature calculations, we removed the rows with missing data.

3 Proposed Method

3.1 Problem Formulation

3.1.1 Data and Notation

Let (O_t, H_t, L_t, C_t) denote the 5-minute Open, High, Low and Close sequence for a given instrument. We construct a feature vector at each index t from price-derived signals:

$$\mathbf{x}_t = [\text{DistSH}_t, \text{DistSL}_t, \text{OB}_t, \text{MA20}_t, \text{MA50}_t, \text{RSI}_t, \text{Diff}_t],$$

where (i) $\text{DistSH}_t = C_t - \text{SH}_t$, $\text{DistSL}_t = C_t - \text{SL}_t$; (ii) $\text{OB}_t \in \{0, 1\}$ indicates order-block proximity; (iii) $\text{MA20}_t, \text{MA50}_t$ are simple moving averages; (iv) RSI_t is the Relative Strength Index; and (v) $\text{Diff}_t = \text{MA20}_t - \text{MA50}_t$. All rolling statistics use only past data up to t (no leakage).

3.2 Training and Testing Splits

We perform a *temporal* split of the time series into non overlapping blocks: the first block for training and the subsequent block for testing. Let $\mathcal{T}_{\text{train}}$ and $\mathcal{T}_{\text{test}}$ denote the index sets corresponding to the training and testing periods, respectively (We define the exact splitting in the upcoming subsections).

- **Training labels** For each training index t , we construct the binary label

$$y_t = \begin{cases} 1, & \text{if } C_{t+h} > C_t, \\ 0, & \text{otherwise,} \end{cases}$$

where C_t is the close at time t and h is the forecast horizon in bars (in our case, $h = 50$, this value is a hyper parameter). These labels are denoted $\{y_t\}$ and are used to fit the classifier f . Since we are accessing the *Close*

prices of the future to calculate the label, there is a chance of data leakage, i.e. while we calculate the labels for the train data, we might access the data which is beyond the training segment (test segment). This issue has been addressed in the upcoming subsections.

- **Test predictions** For testing, at time t , we compute the features from the closing prices available before or at t (for our case, to generate the features, we roll back to at most 50 steps behind to calculate features, like the MA_{50} , the parameters for the moving averages can be controlled) to generate the predicted labels:

$$\hat{y}_t = f(\mathbf{x}_t), \quad t \in \mathcal{T}_{\text{test}}.$$

- **Leakage control at the split:** Since training labels use C_{t+h} , the last h training points would reference the *Close* prices that falls inside the test window. To eliminate any cross-boundary look-ahead, we *discard the first h samples of the test block* before evaluation:

$$\mathcal{T}_{\text{test}} = \{t \in \mathcal{T}_{\text{test}} \mid t \geq t_{\text{test, start}} + h\}.$$

All rolling statistics and features are computed with data $\leq t$ only, and the model selection/evaluation uses the leakage-free set $\mathcal{T}_{\text{test, eval}}$. For train-test splitting, we are first splitting the time series data in a 80:20 ratio for training and testing respectively. Post splitting, we are discarding the first h (in our case $h = 50$) from test data and evaluating the strategy. For all stocks under NIFTY 50, we are directly applying the splitting algorithm to the complete data corresponding to each stocks. But for the Forex pairs, we are breaking down the dataset (maintaining the time order) into chunks of size 3225. So there will be 3225 rows available in each chunk, and this train/test is experimented on each chunk and results are available in the appendix. The reason we are doing this kind of chunking is due to the large time that PDT takes to learn for bigger training data.

3.2.1 Objective

Learn a classifier $f : \mathbb{R}^d \rightarrow \{0, 1\}$ where d is the dimension of the feature vector (7 in our case). We instantiate f as a Permutation Decision Tree (PDT) using the Effort-To-Compress (*ETC*) impurity on the *time-ordered* label sequence at each node. In addition, we use a rule-based execution policy that maps f 's predictions into trades as follows:

- **Entry:** If $\hat{y}_t = f(\mathbf{x}_t) = 1$, initiate a long position at the close C_t (buy the maximum number of shares with available cash); otherwise, remain flat (hold no position and place no orders).
- **Exit:** We decide to sell all our positions if at least one of the following conditions are satisfied:
 - **Trailing stop-loss:** Let t_e be the entry time and running time is defined as follows:

$$M_t = \max_{u \in [t_e, t]} C_u,$$

where t is the current time and t_e is the time at which we had purchased the stocks (Entry condition). For a trail parameter $\lambda \in (0, 1)$, the stop level is (S_t):

$$S_t = (1 - \lambda) M_t,$$

In our use case, we have used the value of $\lambda = 0.005$. This value can be changed in order to control the behavior of the strategy.

Exit occurs at the first t such that $C_t \leq S_t$ (filled at S_t).

- **Model Prediction:** We also exit if the prediction flips, $\hat{y}_t = 0$.
- **Objective function:** We select model and policy parameters (tree depth, node size, λ , h) to *maximize profits*.

3.2.2 ETC Impurity and Split Gain.

For a node containing the ordered labels $S = (y_{t_1}, \dots, y_{t_m})$, let $ETC(S)$ be its *ETC* value. For a candidate split (j, τ) on feature $x^{(j)}$ at threshold τ , let S_L, S_R be the left/right label subsequences (preserving the original temporal order). The *ETC gain* is

$$\text{Gain}(j, \tau) = ETC(S) - \frac{|S_L|}{|S|} ETC(S_L) - \frac{|S_R|}{|S|} ETC(S_R).$$

We choose $(j, \tau) = \arg \max_{(j, \tau)} \text{Gain}(j, \tau)$.

3.2.3 Tree Growth and Stopping

Top-down recursion of the Permutation Decision Tree construction continues until any of the following holds: (i) $ETC(S) = 0$ (homogeneous), (ii) node size $< n_{\min}$, (iii) depth $= D_{\max}$.

3.2.4 Method to calculate *ETC*.

At any node of the tree, we take the *time-ordered* sequence of labels that reach that node, e.g.

$$S = (y_{t_1}, y_{t_2}, \dots, y_{t_m}), \quad y_{t_i} \in \{0, 1\}.$$

We compute the Effort-To-Compress (*ETC*) of this sequence using the Non-Sequential Recursive Pair Substitution (NSRPS) [17] procedure: repeatedly (a) count all consecutive pairs, (b) among the most frequent pairs pick the one that *appears first* when scanning left-to-right, and (c) replace all *non-overlapping* occurrences of that pair with a new symbol. The *ETC* value is the number of such iterations needed to make the sequence homogeneous. This is the impurity at that node; when we try a split, we compare the node’s *ETC* to the (size-weighted) *ETC* of the left/right label subsequences to get the *ETC* Gain.

Worked example (8 consecutive 5-minute labels). Suppose the label sequence for 8 successive 5-minute bars is

$$S = [0, 0, 1, 0, 1, 1, 0, 1].$$

Applying NSRPS step-by-step (ties broken by the first *occurring* max-frequency pair; replacements are non-overlapping):

1. *Iter 1.* Pair counts: $(0, 1) \times 3$, $(1, 0) \times 2$, $(0, 0) \times 1$, $(1, 1) \times 1$.
First max pair encountered is $(0, 1)$. Replace all non-overlapping $(0, 1) \Rightarrow 2$:
 $[0, 0, 1, 0, 1, 1, 0, 1] \Rightarrow [0, 2, 2, 1, 2]$.
2. *Iter 2.* Pair counts: all frequency 1; first encountered is $(0, 2)$. Replace $(0, 2) \Rightarrow 3$:
 $[0, 2, 2, 1, 2] \Rightarrow [3, 2, 1, 2]$.
3. *Iter 3.* First encountered max pair is $(3, 2)$. Replace $(3, 2) \Rightarrow 4$:
 $[3, 2, 1, 2] \Rightarrow [4, 1, 2]$.
4. *Iter 4.* First encountered max pair is $(4, 1)$. Replace $(4, 1) \Rightarrow 5$:
 $[4, 1, 2] \Rightarrow [5, 2]$.
5. *Iter 5.* Only pair is $(5, 2)$. Replace $(5, 2) \Rightarrow 6$:
 $[5, 2] \Rightarrow [6]$.

The sequence is now homogeneous, so $ETC(S) = 5$. This is exactly the computation we perform on the *entire* time-ordered label set at the root (and on each node’s label subsequence thereafter) when evaluating splits via *ETC* Gain.

3.3 Permutation Decision Tree

A Permutation Decision Tree (PDT) is a decision-tree-based algorithm that measures the *uniformity* of target labels using a novel metric called *Effort To Compress (ETC)* [15, 16] instead of traditional criteria (e.g., Gini impurity). By focusing on *ETC*, a PDT locates splits that make the target labels (0s and 1s) in each node more “compressible,” revealing threshold-based patterns that conventional splitting criteria may not detect.

Key Concepts in PDT:

- **Effort To Compress (*ETC*):** A measure of how many pair-replacement iterations are needed to compress a label sequence into a single repeated symbol. The fewer the iterations, the more uniform (or “pure”) the sequence is. See Algorithm 1. Unlike Shannon entropy and Gini impurity, *ETC* does not assume independent and identical distribution (i.i.d). This makes *ETC* as an impurity measure ideal for datasets having temporal dependencies across data instances [14].
- **Split Criterion (*ETC* Gain):** When choosing a threshold on a given feature, we compute the drop in *ETC* (weighted by the sizes of the child nodes) from the parent node to its left/right children. A higher reduction (i.e., higher *ETC* Gain) implies a more effective split. See Algorithm 2.

Thus, PDT explicitly seeks splits that yield subsets of target labels that can be “compressed” more easily. This can reveal threshold-based patterns that might not be captured by conventional impurity metrics.

Once we determine the “best threshold” for a feature, the next step is to see which feature *overall* (among many) yields the highest *ETC* Gain. This is handled by FIND_BEST_FEATURE, shown in Algorithm 3.

Finally, the overall PDT is constructed *recursively* by BUILD_PDT (Algorithm 4). We keep splitting until labels become uniform or the maximum depth of the tree (stopping criteria) is met.

Algorithm 1: CALCULATEETC – Computes the *ETC* Value of a Sequence.

Require: A sequence *seq* (list or string of digits)

Ensure: ETC(*seq*): number of pair-replacement iterations until uniformity

```
1: itrs  $\leftarrow$  0
2: Convert all elements in seq to integers
3: max_symbol  $\leftarrow$  max(seq)
4: while |set(seq)| > 1 do
5:   Count all consecutive pairs (seq[i], seq[i + 1]) for i = 0, 1, ..., |seq| - 2
6:   highest_freq_pair  $\leftarrow$  pair with max frequency
7:   max_symbol  $\leftarrow$  max_symbol + 1
8:   new_symbol  $\leftarrow$  max_symbol
9:   temp_seq  $\leftarrow$  []
10:  i  $\leftarrow$  0
11:  while i < |seq| do
12:    if i < |seq| - 1 and (seq[i], seq[i + 1]) = highest_freq_pair then
13:      Append new_symbol to temp_seq
14:      i  $\leftarrow$  i + 2
15:    else
16:      Append seq[i] to temp_seq
17:      i  $\leftarrow$  i + 1
18:    end if
19:  end while
20:  seq  $\leftarrow$  temp_seq
21:  itrs  $\leftarrow$  itrs + 1
22: end while
23: return itrs
```

3.4 Framework Using PDT for Stock Classification

To classify short-term stock movements (up or down), we integrate PDT within the following pipeline (illustrated in Figure 1):

1. Building the PDT Model:

- Calculate *ETC* of the entire (parent) label set.
- Search thresholds for each feature to find the best *ETC Gain* split (Algorithms 2 and 3).
- Recursively grow left/right child nodes until purity or maximum depth (Algorithm 4).

2. Prediction: For a new feature vector (e.g., the next 5-minute candle and indicators):

- Traverse the PDT from the root, comparing feature values to thresholds.
- Stop at a leaf, which outputs $\hat{y} \in \{0, 1\}$.
- If $\hat{y} = 1$, we anticipate a price increase; if $\hat{y} = 0$, we expect no increase (or a drop).

3.5 Evaluation Metrics

To measure the performance and effectiveness of the proposed trading strategy, we use the following evaluation metrics:

3.5.1 Growth (%)

The growth measures the overall percentage gain or loss achieved during the trading period. It is calculated as:

$$\text{Growth \%} = \frac{V_{\text{final}} - V_{\text{initial}}}{V_{\text{initial}}} \times 100,$$

where V_{final} is the final portfolio value at the end of the trading period and V_{initial} (in our case $V_{\text{initial}}=10,000$ units of currency, INR for Indian stocks and USD for Forex pairs) is the initial portfolio value.

Algorithm 2: *ETC_Gain* – Finds the Best Threshold Split for a Single Feature.

Require: *data*: 2D array of shape (n, d)
 labels: 1D array of length n
 feature_index: Index of feature to split on
Ensure: $(best_gain, best_threshold)$

- 1: Convert *labels* to integer array
- 2: $totalETC \leftarrow \text{CALCULATEETC}(labels)$
- 3: $feature_values \leftarrow data[:, feature_index]$
- 4: $unique_vals \leftarrow$ sorted unique values of *feature_values*
- 5: **if** $|unique_vals| = 1$ **then**
- 6: **return** $(-\infty, \text{None})$
- 7: **end if**
- 8: $thresholds \leftarrow$ midpoints between consecutive elements of *unique_vals*
- 9: $best_gain \leftarrow -\infty$
- 10: $best_threshold \leftarrow \text{None}$
- 11: **for each** threshold $t \in thresholds$ **do**
- 12: $left_mask \leftarrow feature_values \leq t$
- 13: $right_mask \leftarrow \text{not } left_mask$
- 14: $left_labels \leftarrow labels[left_mask]$
- 15: $right_labels \leftarrow labels[right_mask]$
- 16: $etcL \leftarrow \text{CALCULATEETC}(left_labels)$ if $|left_labels| > 0$ else 0
- 17: $etcR \leftarrow \text{CALCULATEETC}(right_labels)$ if $|right_labels| > 0$ else 0
- 18: $wL \leftarrow \frac{|left_labels|}{|labels|}$
- 19: $wR \leftarrow \frac{|right_labels|}{|labels|}$
- 20: $weightedETC \leftarrow wL \times etcL + wR \times etcR$
- 21: $gain \leftarrow totalETC - weightedETC$
- 22: **if** $gain > best_gain$ **then**
- 23: $best_gain \leftarrow gain$
- 24: $best_threshold \leftarrow t$
- 25: **end if**
- 26: **end for**
- 27: **return** $(best_gain, best_threshold)$

Algorithm 3: *FIND_BEST_FEATURE* – Identifies the Feature and Threshold with Maximum *ETC Gain*.

Require: *data*: 2D array of shape (n, d)
 labels: 1D array of length n
Ensure: $(bestFeature, bestThreshold)$

- 1: Convert *data*, *labels* to arrays (if not already)
- 2: $numFeatures \leftarrow$ number of columns in *data*
- 3: $bestFeature \leftarrow \text{None}$
- 4: $bestThreshold \leftarrow \text{None}$
- 5: $bestGain \leftarrow -\infty$
- 6: **for** $f_idx = 0$ to $numFeatures - 1$ **do**
- 7: $(gain, threshold) \leftarrow \text{ETC_GAIN}(data, labels, f_idx)$
- 8: **if** $gain > bestGain$ **then**
- 9: $bestGain \leftarrow gain$
- 10: $bestFeature \leftarrow f_idx$
- 11: $bestThreshold \leftarrow threshold$
- 12: **end if**
- 13: **end for**
- 14: **return** $(bestFeature, bestThreshold)$

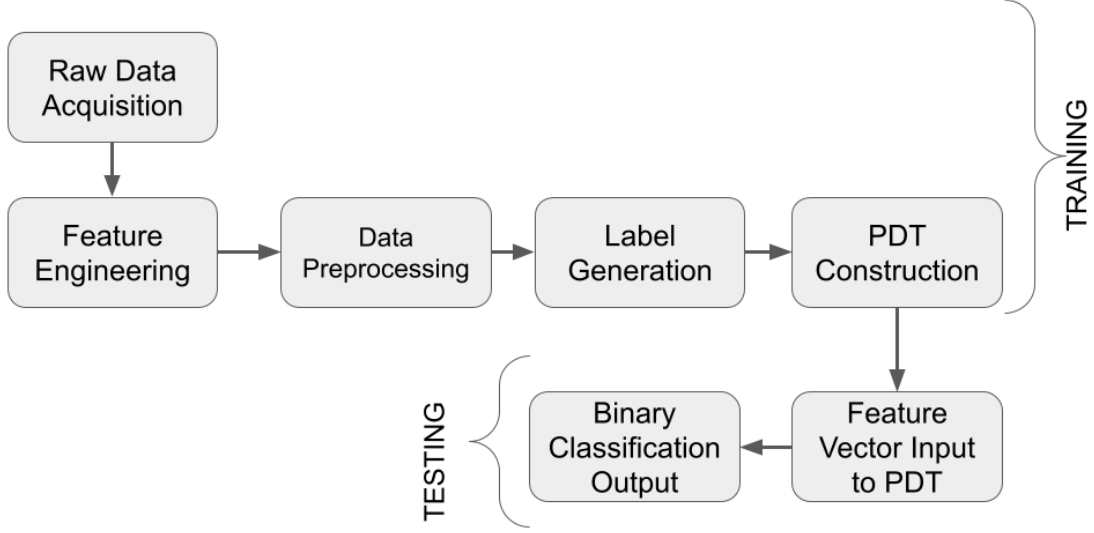


Figure 1: Architecture Diagram of the Proposed PDT-Based Stock Classification Pipeline. Data is collected and preprocessed, then used to build a Permutation Decision Tree, which predicts stock movements for future intervals.

3.5.2 Maximum Drawdown (%)

The maximum drawdown quantifies the largest percentage decline in the portfolio from its peak to its lowest point during the trading period. It is calculated as:

$$\text{Maximum Drawdown} = \max \left(\frac{V_{\text{peak}} - V_t}{V_{\text{peak}}} \times 100 \right),$$

where V_{peak} is the highest portfolio value observed at or before time t and V_t is the portfolio value at time t .

These metrics collectively provide a comprehensive assessment of both *profitability* and *risk management* for any trading approach driven by PDT-based classification.

4 Experimental Setup

This section explains the procedure used to simulate trading strategies based on the predictions generated by the PDT model.

Algorithm 4: BUILD_PDT – Recursively Constructs a Permutation Decision Tree.

Require: *data*: Feature matrix, *labels*: target labels
 depth: current depth, *maxDepth*: maximum allowed depth
 pbar: optional progress tracker
Ensure: A PDT node (dictionary) or a leaf label

```
1: Convert data and labels to NumPy arrays
2: if |data| = 0 then
3:   if pbar ≠ None then
4:     pbar.update(1)
5:   end if
6:   return None
7: end if
8: if all labels in labels are identical then
9:   if pbar ≠ None then
10:    pbar.update(1)
11:   end if
12:   return that identical label
13: end if
14: if depth ≥ maxDepth then
15:   if pbar ≠ None then
16:    pbar.update(1)
17:   end if
18:   Return majority label in labels
19: end if
20: (bestFeature, bestThreshold) ← FIND_BEST_FEATURE(data, labels)
21: if bestThreshold = None then
22:   if pbar ≠ None then
23:    pbar.update(1)
24:   end if
25:   Return majority label in labels
26: end if
27: tree ← {feature_index : bestFeature, threshold : bestThreshold, children : {}}
28: leftMask ← data[:, bestFeature] ≤ bestThreshold
29: rightMask ← not leftMask
30: leftData, leftLabels ← data[leftMask], labels[leftMask]
31: rightData, rightLabels ← data[rightMask], labels[rightMask]
32: tree.children.left ← BUILD_PDT(leftData, leftLabels, depth + 1, maxDepth, pbar)
33: tree.children.right ← BUILD_PDT(rightData, rightLabels, depth + 1, maxDepth, pbar)
34: if pbar ≠ None then
35:   pbar.update(1)
36: end if
37: return tree
```

4.1 Key Components of the Simulation

The simulation involves the following steps:

1. **Initializing Portfolio:** Start with an initial balance (in our setup, we have started with 10,000INR for indian stocks and 10,000USD for forex pairs) and no positions.
2. **Iterative Trading:** For each time interval:
 - Evaluate the model's prediction for the current interval.
 - Execute buy, sell, or hold decisions based on the trading rules defined in Section 3.2.1.
3. **Risk Management:** Use a trailing stop-loss mechanism (defined in Section 3.2.1) to limit losses and lock in profits.
4. **Portfolio Value Update:** Update the portfolio balance and positions after each trade.

4.2 Structure of Results DataFrame

The **results_df** is a key input to the simulation algorithm. It contains the following columns:

- **Datetime:** The timestamp of the corresponding stock price data.
- **Actual:** The actual label indicating whether the price increased (1) or decreased (0).
- **Predicted:** The label predicted by the PDT model (1 for increase, 0 for decrease).
- **Close:** The closing price of the stock at the specified timestamp.

An example of the **results_df** is shown in Table 4, which illustrates the structure of this DataFrame.

Table 4: Example Structure of Results DataFrame (results_df)

Datetime	Actual	Predicted	Close
2024-10-25 13:25:00	0	0	1487.87
2024-10-25 13:30:00	1	1	1488.50
2024-10-25 13:35:00	0	0	1488.37
2024-10-25 13:40:00	1	1	1488.45
2024-10-25 13:45:00	0	0	1491.30

5 Results and Discussions

In this section, we describe how each model (PDT, LSTM, RNN) and the Buy-and-Hold baseline was evaluated using our *simulation process* (Algorithm 5), followed by a detailed discussion of the Permutation Decision Tree (PDT) results. For PDT, LSTM and RNN, we have evaluated their performances across all 50 stocks that come under the NIFTY 50 index and aggregated the stocks performance by taking an average for each model. Over and above this, we also have tested PDT on Forex pairs EURUSD and XAUUSD as mentioned in the begining of this paper. The results for each stock under NIFTY 50 for PDT is available in the appendix.

5.1 Simulation Algorithm (for All Models)

Before reviewing individual model performance, we provide the core simulation algorithm (Algorithm 5) that executes trades based on *Predicted* signals.

Algorithm 5: SIMULATION PROCESS – Executes Buy/Sell Decisions Based on PDT Predictions.

Require: *stock_symbol*, *results_df* with columns {Datetime, Actual, Predicted, Close}, Initial balance B_0 , Trailing stop-loss δ

Ensure: Array of portfolio values $P = [P_1, P_2, \dots, P_T]$ and final metrics

```

1:  $B \leftarrow B_0$  // current balance
2:  $positions \leftarrow 0$ 
3:  $trade\_count \leftarrow 0$ 
4:  $successful\_trades \leftarrow 0$ 
5:  $P \leftarrow []$  // list of portfolio values over time
6:  $entry\_price \leftarrow \text{None}$ 
7:  $trailing\_stop \leftarrow \text{None}$ 
8: for  $t = 1$  to  $T$  do
9:    $pred \leftarrow results\_df[t][\text{Predicted}]$ 
10:   $price \leftarrow results\_df[t][\text{Close}]$ 
11:  if  $pred = 1 \wedge positions = 0$  then
12:     $positions \leftarrow \lfloor B/price \rfloor$ 
13:     $entry\_price \leftarrow price$ 
14:     $B \leftarrow B - (positions \times price)$ 
15:     $trailing\_stop \leftarrow entry\_price \times (1 - \delta)$ 
16:  end if
17:  if  $positions > 0$  then
18:     $trailing\_stop \leftarrow \max(trailing\_stop, price \times (1 - \delta))$ 
19:    if  $price \leq trailing\_stop \vee pred = 0$  then
20:       $profit \leftarrow positions \times (price - entry\_price)$ 
21:       $B \leftarrow B + (positions \times price)$ 
22:       $trade\_count \leftarrow trade\_count + 1$ 
23:       $successful\_trades \leftarrow successful\_trades + \mathbb{I}_{(profit > 0)}$ 
24:       $positions \leftarrow 0$ 
25:    end if
26:  end if
27:   $P_t \leftarrow B + (positions \times price)$ 
28:  Append  $P_t$  to  $P$ 
29: end for
30: if  $positions > 0$  then
31:   $profit \leftarrow positions \times (price - entry\_price)$ 
32:   $B \leftarrow B + (positions \times price)$ 
33:   $positions \leftarrow 0$ 
34: end if
35: Total Return %  $\leftarrow \left( \frac{B - B_0}{B_0} \right) \times 100$ 
36: Trading Accuracy %  $\leftarrow \left( \frac{successful\_trades}{trade\_count} \right) \times 100$ 
37: return  $P$  // Full series of portfolio values

```

5.2 Permutation Decision Tree (PDT) Performance

We now present the results of applying the **Permutation Decision Tree (PDT)** model. While this shorter time frame captures only short-term trends, it demonstrates the PDT's capability to adapt quickly to market fluctuations.

Key Observations:

- **NIFTY 50 Performance aggregation**
 - **PDT Strategy:**
 - * **Average Return (Testing period):** 1.1802% per stock.
 - **Buy-and-Hold Strategy:**
 - * **Average Return (Testing period):** -2.29% per stock.
- **EURUSD Performance**
 - **PDT Strategy:**
 - * **Return (Testing period):** 2.3105%.
 - **Buy-and-Hold Strategy:**
 - * **Return (Testing period):** 1.6246%.
- **XAUUSD Performance**
 - **PDT Strategy:**
 - * **Return (Testing period):** 0.0006%.
 - **Buy-and-Hold Strategy:**
 - * **Return (Testing period):** 0.0021%.

Performance Analysis: The PDT-based approach demonstrated **superior returns** compared to buy-and-hold for NIFTY 50 stocks on an average and in EURUSD, indicating its robustness and adaptability for short-term trading. The inclusion of a trailing stop-loss in Algorithm 5 helped to curtail losses, preserving capital during sudden market reversals. Even though PDT underperformed in XAUUSD, we still made a profit (positive return).

Limitations Due to Limited Data: Because the evaluation spanned only around 3 months for NIFTY 50 and around 5 months for the Forex pairs, these findings mainly reflect short-term performance.

Visual Representation of Portfolio Performance: Figure 2 compares the **PDT-based strategy** and *buy-and-hold* for NIFTY 50 stocks on an average in terms of portfolio value growth. PDT consistently outperforms, incurring far fewer drawdowns.

Drawdown Analysis (PDT vs. Buy-and-Hold): Figure 3 illustrates the drawdown curves for the NIFTY 50 stocks on an average.

5.3 LSTM and RNN Performance

We begin by evaluating two deep learning models Long Short-Term Memory (LSTM) and Recurrent Neural Network (RNN)—on the NIFTY 50 stocks dataset. We have used these models since they are commonly used in stock price predictions.

Hyperparameters and Implementation: The hyperparameters for the LSTM and RNN models are chosen to balance computational complexity and performance:

- **Number of Units (50):** Provides sufficient capacity for learning patterns.
- **Dense Layer (10 units):** Additional layer to capture complex relationships.
- **Activation Functions:**
 - **ReLU:** Used in the dense layer.
 - **Sigmoid:** Applied to the output layer for binary classification.
- **Loss Function:** Binary cross-entropy.
- **Optimizer:** Adam.
- **Epochs:** 50.

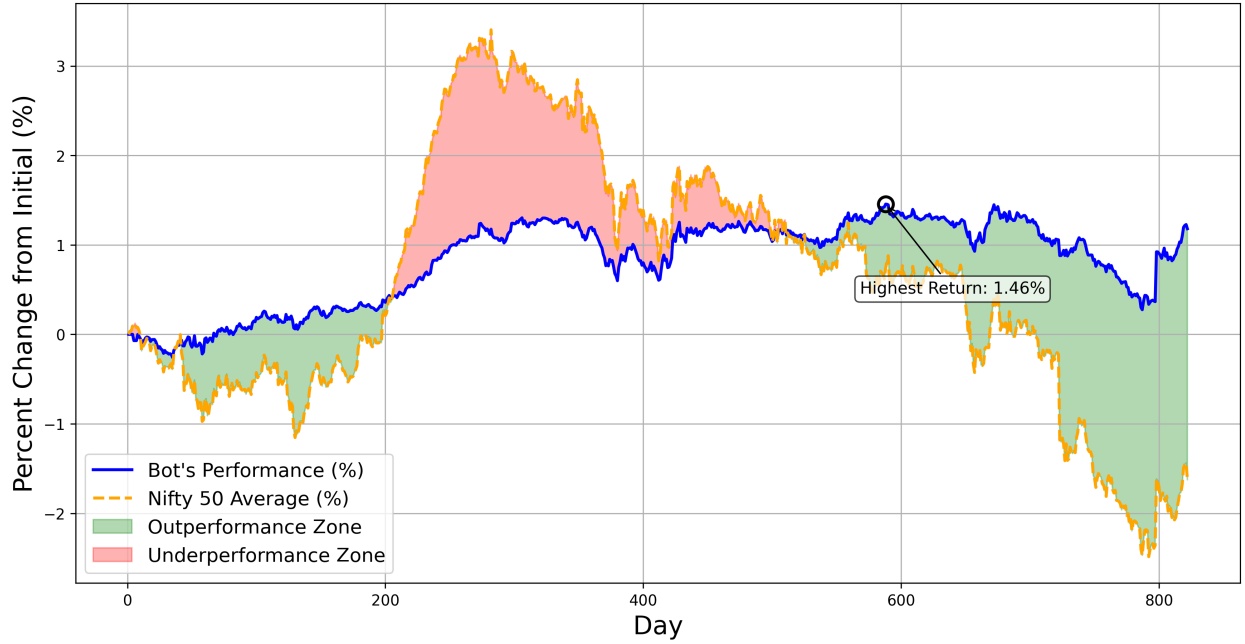


Figure 2: Portfolio performance comparison (PDT vs. Buy-and-Hold) over 3 testing-month period. PDT shows *steady growth*, whereas buy-and-hold experiences significant losses.

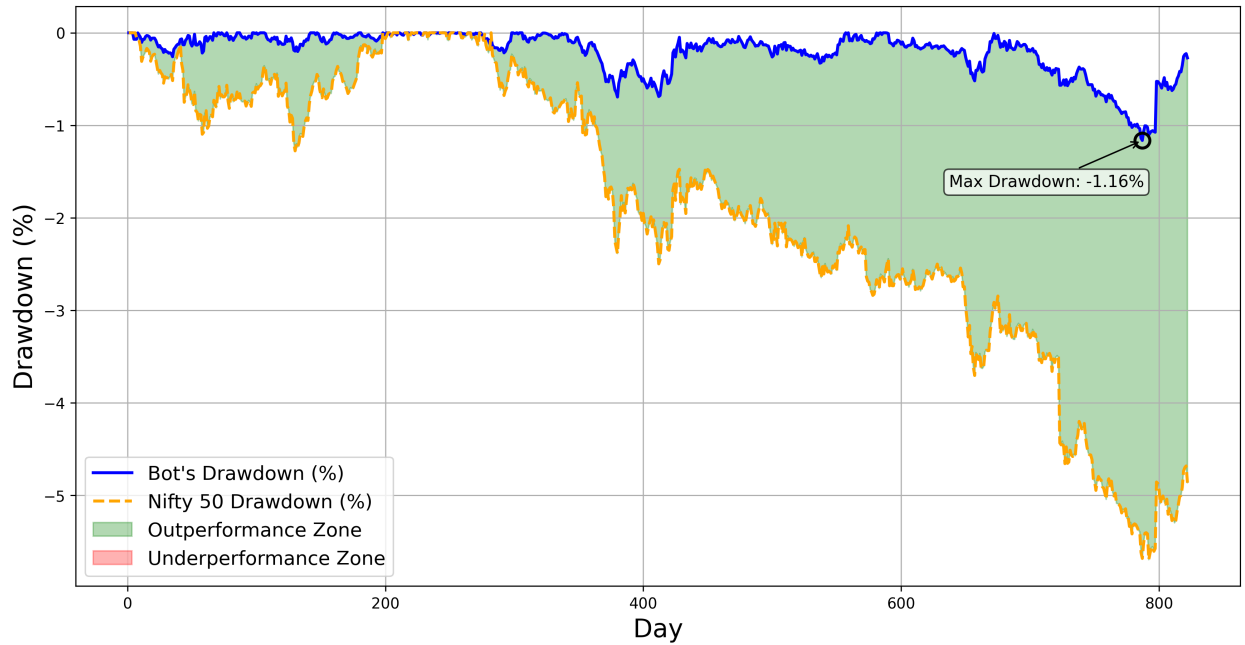


Figure 3: Drawdown for PDT vs. Buy-and-Hold.

The training processes for LSTM and RNN are given in Algorithms 6 and 7.

Drawdown and Portfolio Performance: Figures 4 and 5 illustrate the drawdown curves for LSTM and RNN respectively for the NIFTY 50 stocks on an average, while Figures 6 and 7 compare each model's portfolio value to that of a naive buy-and-hold approach.

Overall, LSTM and RNN (NIFTY 50 stocks, on an average):

Algorithm 6: TRAINLSTM – Builds and Trains an LSTM Model

Require: Feature matrix X , Target labels y , Train-test split index $split_idx$

Ensure: Trained LSTM model, predicted test labels y_pred

- 1: Reshape X into a 3D tensor: (samples, timesteps = 1, features)
 - 2: Split X, y into training and testing sets
 - 3: Build an LSTM model:
 - One LSTM layer with 50 units
 - One Dense layer with 10 units and ReLU activation
 - Output layer with 1 unit and Sigmoid activation
 - 4: Compile using Adam and binary cross-entropy loss
 - 5: Train for 50 epochs, batch size = 32, learning rate = 0.001
 - 6: Predict probabilities on the test set, threshold at 0.5 to get binary labels
 - 7: **return** $model, y_pred$
-

Algorithm 7: TRAINRNN – Builds and Trains a Simple RNN

Require: Feature matrix X , Target labels y , Train-test split index $split_idx$

Ensure: Trained RNN model, predicted test labels y_pred

- 1: Reshape X into a 3D tensor: (samples, timesteps = 1, features)
 - 2: Split X, y into training and testing sets
 - 3: Build an RNN model:
 - One SimpleRNN layer with 50 units
 - One Dense layer with 10 units and ReLU activation
 - Output layer with 1 unit and Sigmoid activation
 - 4: Compile using Adam and binary cross-entropy loss
 - 5: Train for 50 epochs, batch size = 32, learning rate = 0.001
 - 6: Predict probabilities on the test set, threshold at 0.5 to get binary labels
 - 7: **return** $model, y_pred$
-

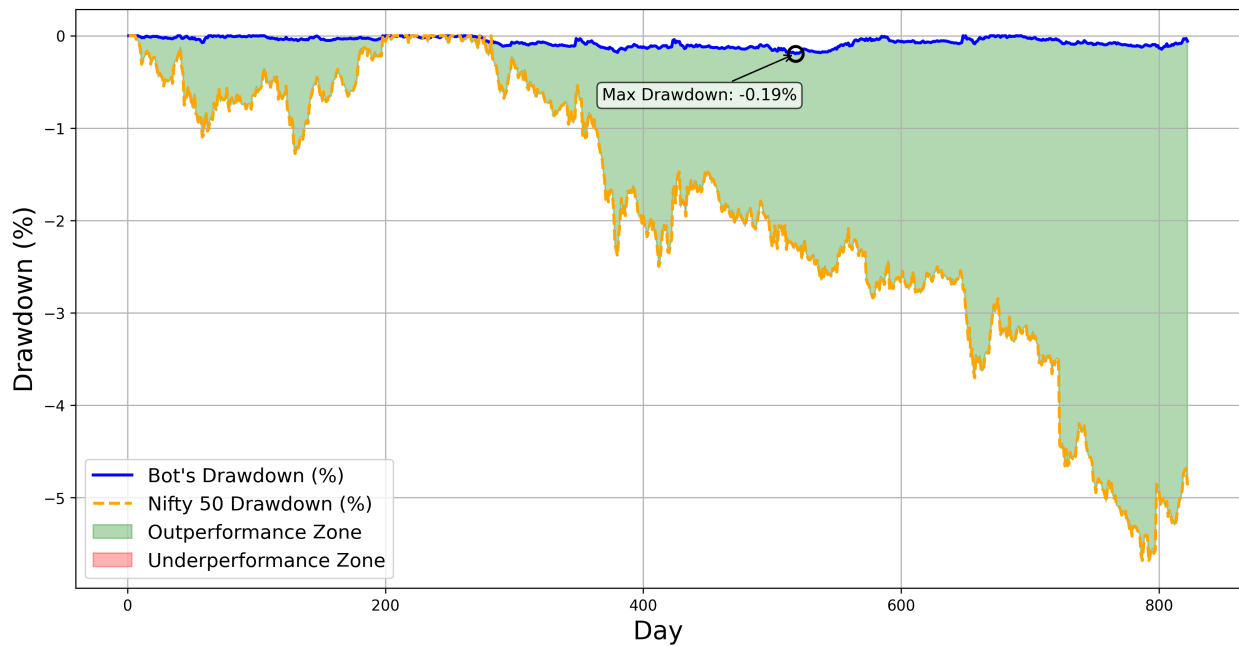


Figure 4: Drawdown for LSTM.

- Provide improved returns over Buy-and-Hold in certain time windows.

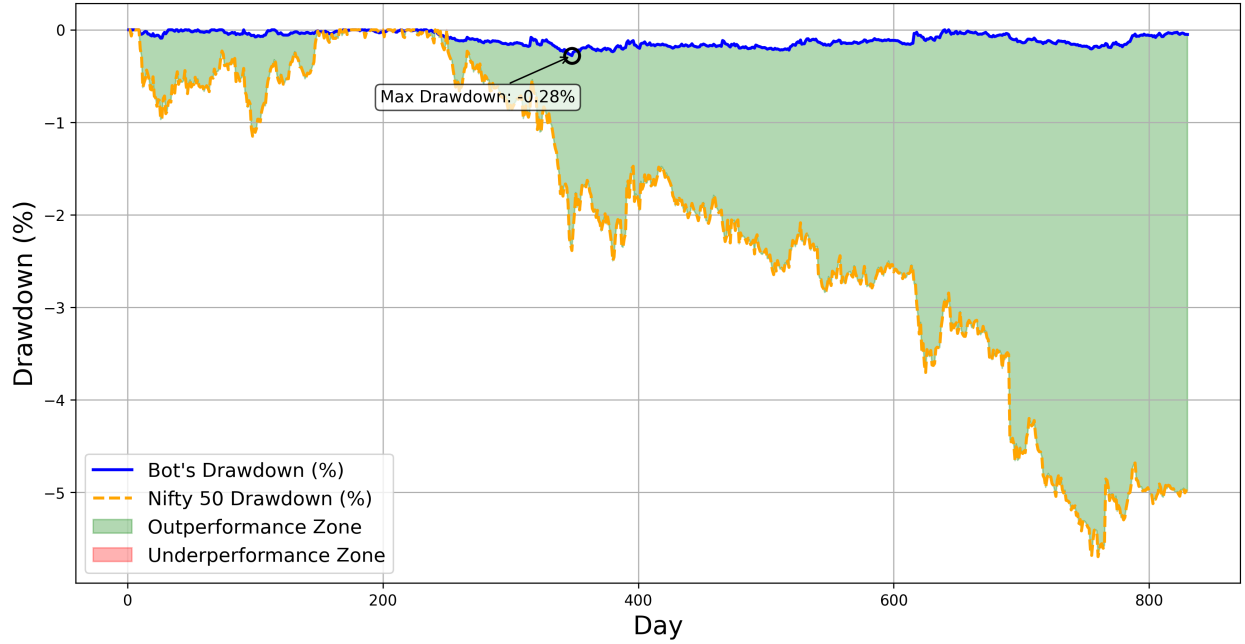


Figure 5: Drawdown for RNN.

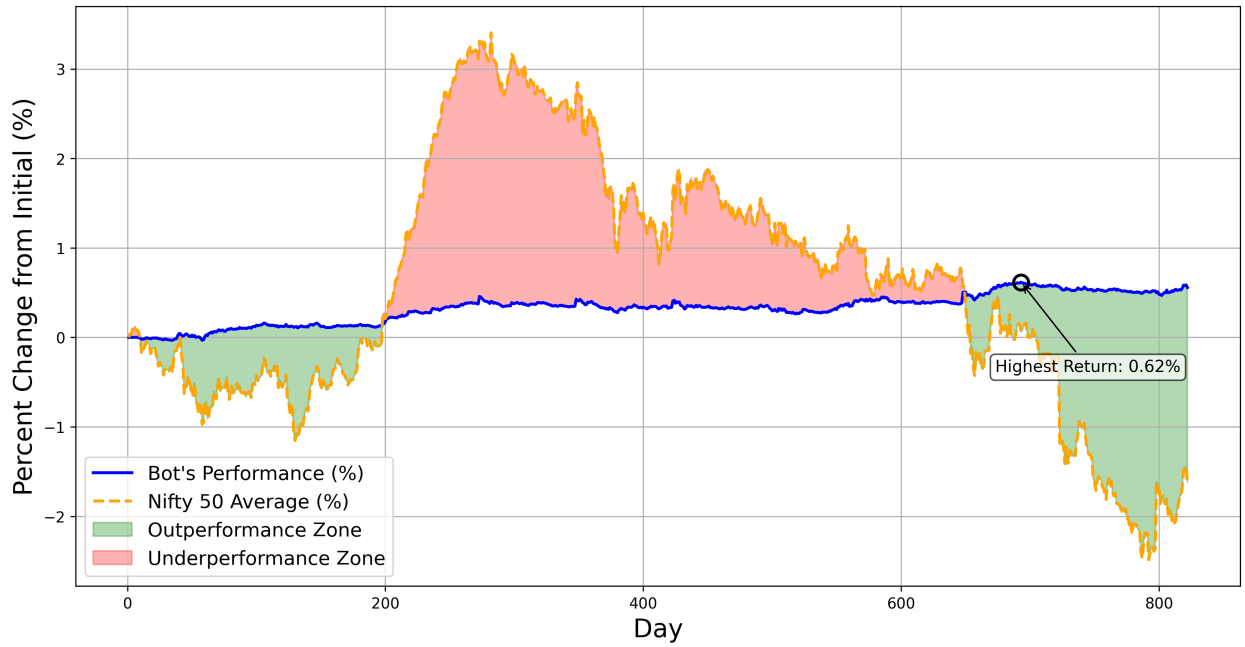


Figure 6: Portfolio Value: LSTM vs. Buy-and-Hold.

- Are highly data-dependent and can struggle in sudden market shifts.

5.4 Comparison with Baseline Models

In addition to Buy-and-Hold, we benchmark PDT against **LSTM** and **RNN** (Table 5).

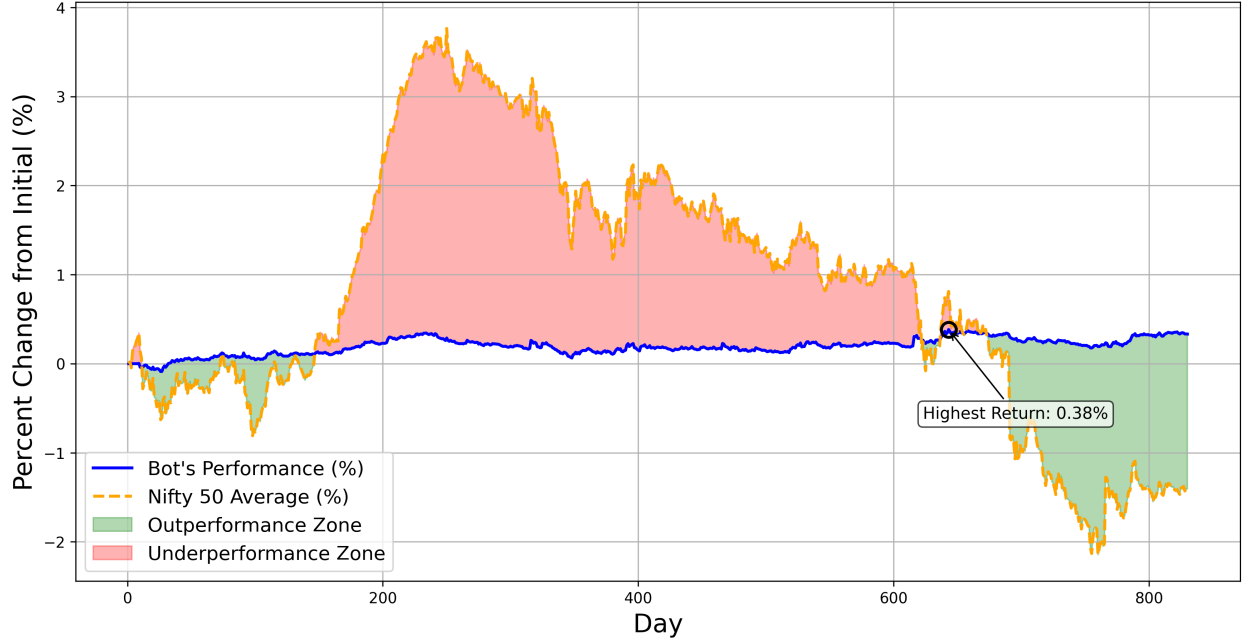


Figure 7: Portfolio Value: RNN vs. Buy-and-Hold.

Table 5: Performance Comparison of PDT, LSTM, RNN, and Buy-and-Hold.

Model	Testing Period Growth (%)	Max Drawdown (%)
PDT	1.1802	1.16
LSTM	0.557	0.19
RNN	0.5896	0.28
Buy-and-Hold	-2.29	5.6955

5.5 Discussions

- **PDT vs. LSTM/RNN:** PDT exhibits higher returns (1.1802% on NIFTY 50 on an average), suggesting that its *Effort To Compress (ETC)* metric identifies clearer thresholds for stock movement. LSTM/RNN (0.557% and 0.5896% on an average, respectively) require larger datasets to fully capture temporal patterns.
- **Risk/Reward Profile:** The trailing stop-loss used in our simulation helps PDT lock in gains and limit downside, whereas LSTM/RNN, in the same simulation, do not show as effective a drawdown control.
- **Buy-and-Hold Vulnerability:** With a negative testing period growth of -2.29% and large drawdowns, this passive strategy struggles in a volatile 3 month market window.
- **Overall Conclusion:** In a short 3 month study, PDT outperforms both deep learning baselines and buy-and-hold, but more extensive tests over multiple market cycles are advised to confirm long-term robustness.

6 Limitations and Future Research Directions

Permutation Decision Trees (PDT) have shown promising results in stock price prediction; however, several limitations and strengths must be highlighted for balanced evaluation. PDT heavily relies on quality input features; noisy or incomplete data significantly reduces performance. PDTs cannot be incrementally updated. With new data, the entire tree must be reconstructed, limiting adaptability in dynamic markets. Future research should address these limitations, specifically by incorporating dynamic updating of PDT, inclusion of transaction costs and slippage, and better modeling of sequential dependencies.

7 Conclusion

In this 3 month study, the Permutation Decision Tree (PDT) demonstrates a superior performance profile compared to LSTM and RNN models. PDT exhibits higher returns suggesting that its Effort To Compress (*ETC*) metric is more effective in identifying key thresholds for stock movement within short-term, high-frequency data. In contrast, LSTM and RNN models appear to require substantially larger datasets to effectively capture the temporal patterns necessary for accurate short-term predictions. The trailing stop-loss mechanism implemented in our simulation proves beneficial for PDT, enabling it to secure gains and mitigate downside risk. Furthermore, a passive buy-and-hold strategy demonstrates significant vulnerability in the volatile market conditions of the testing period, resulting in a negative growth of -2.25% and substantial drawdowns. However, given the short timeframe, further research is recommended. Specifically, more extensive testing across multiple market cycles is necessary to validate the long-term robustness and generalizability of the PDT-based trading system.

Code Availability

The codes used in this research paper are available in the following GitHub repository: <https://github.com/i-to-the-power-i/pdt-stocks>

Acknowledgments

Harikrishnan N. B. gratefully acknowledges the financial support provided by the New Faculty Seed Grant (NFSG/GOA/2024/G0906) from BITS Pilani, K. K. Birla Goa Campus.

Appendix

7.1 Comparison of PDT vs Buy & Hold

The difference in returns $\Delta = \text{PDT Return} - \text{Buy \& Hold Return}$ for each stock is shown below.

Table 6: PDT vs Buy & Hold Returns for NIFTY 50 Stocks.

Stock	PDT Return (%)	Buy & Hold Return (%)	Difference (%)
ASIANPAINT.NS	0.88	-0.20	1.08
BRITANNIA.NS	5.43	3.11	2.32
CIPLA.NS	0.02	-5.23	5.25
EICHERMOT.NS	5.61	1.93	3.68
NESTLEIND.NS	4.39	2.28	2.11
GRASIM.NS	-5.69	-5.63	-0.06
HEROMOTOCO.NS	3.09	-1.96	5.05
HINDALCO.NS	0.38	-4.53	4.91
HINDUNILVR.NS	4.09	3.19	0.90
ITC.NS	0.27	-8.11	8.38
LT.NS	-0.78	-3.46	2.68
M&M.NS	0.17	0.84	-0.67
RELIANCE.NS	2.25	1.39	0.86
TATACONSUM.NS	4.86	7.07	-2.21
TATAMOTORS.NS	7.93	4.25	3.68
TATASTEEL.NS	-1.42	-8.99	7.57
WIPRO.NS	-0.21	-3.81	3.60
APOLLOHOSP.NS	0.06	-7.88	7.94
DRREDDY.NS	-4.06	-2.90	-1.16
TITAN.NS	2.12	1.18	0.94
SBIN.NS	-0.39	-6.64	6.25
SHRIRAMFIN.NS	-2.73	-9.19	6.46
BPCL.NS	0.76	-7.97	8.73
KOTAKBANK.NS	1.90	-1.87	3.77
INFY.NS	3.00	2.33	0.67

Stock	PDT Return (%)	Buy & Hold Return (%)	Difference (%)
BAJFINANCE.NS	3.06	5.19	-2.13
ADANIENT.NS	-0.23	-5.35	5.12
SUNPHARMA.NS	-0.14	-6.82	6.68
JSWSTEEL.NS	2.55	-0.42	2.97
HDFCBANK.NS	-0.88	-9.06	8.18
BAJAJFINSV.NS	0.71	7.73	-7.02
ULTRACEMCO.NS	0.00	-7.56	7.56
DIVISLAB.NS	3.03	-1.81	4.84
MARUTI.NS	0.00	7.29	-7.29
NTPC.NS	0.19	-7.52	7.71
HCLTECH.NS	4.02	-4.30	8.32
HDFCLIFE.NS	1.61	-3.33	4.94
COALINDIA.NS	-2.58	-3.27	0.69
TECHM.NS	4.44	-3.88	8.32
GAIL.NS	3.42	-7.46	10.88
INDUSINDBK.NS	1.78	0.20	1.58
UPL.NS	6.87	6.68	0.19
SHREECEM.NS	0.00	-3.57	3.57
POWERGRID.NS	-0.66	-5.63	4.97
DABUR.NS	2.54	2.35	0.19
ACC.NS	1.78	-7.47	9.25
BAJAJ-AUTO.NS	0.96	-3.03	3.99
CYIENTDLM.NS	-1.93	-12.72	10.79
BHARTIARTL.NS	-2.11	1.39	-3.50
ZEEL.NS	-1.35	-1.33	-0.02

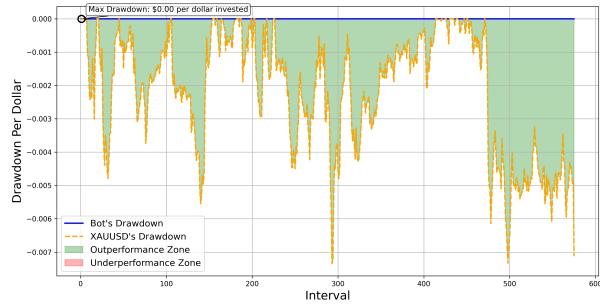
With this table, we can infer that most of the difference is positive which shows that PDT has actually out performed most of the stocks under NIFTY 50 index. In cases where PDT has underperformed with respect to the stocks under NIFTY 50, there are also cases where it still has made profits.

7.2 PDT Profit vs Buy & Hold Profit (Contiguous Chunks)

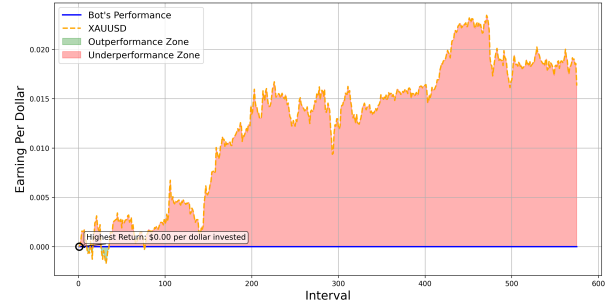
Table 7: PDT Profit vs Buy & Hold Profit for XAUUSD Chunks.

Chunk	PDT Profit	Buy & Hold Profit
1	0.000 00	0.016 33
2	0.011 64	0.029 74
3	0.014 22	0.038 07
4	0.006 22	0.024 91
5	0.009 58	-0.019 24
6	-0.011 63	-0.014 69
7	-0.005 60	0.000 98
8	-0.007 97	-0.012 08

7.3 Drawdown and Portfolio Charts for XAUUSD Chunks

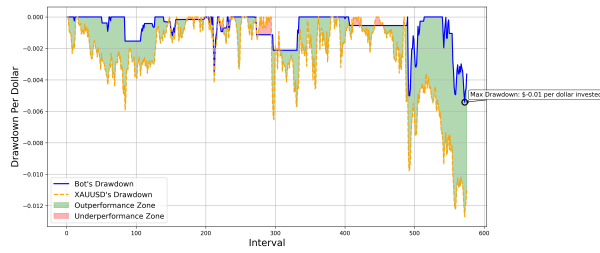


(a) Drawdown

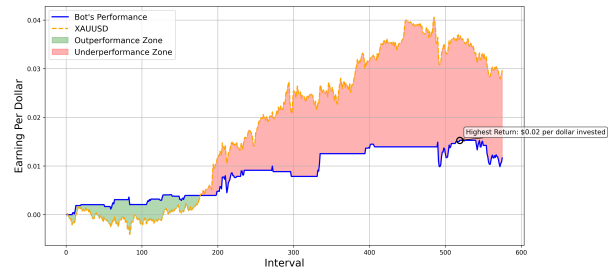


(b) Portfolio

Figure 8: Chunk 1: Drawdown and Portfolio

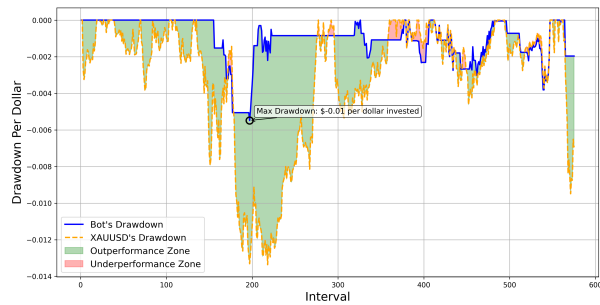


(a) Drawdown

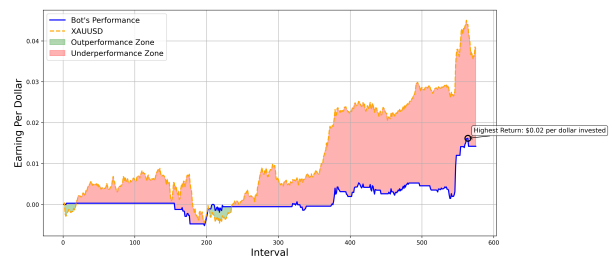


(b) Portfolio

Figure 9: Chunk 2: Drawdown and Portfolio

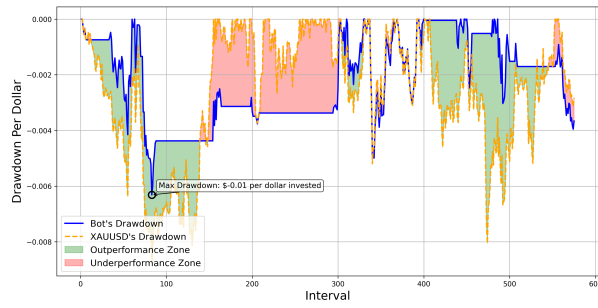


(a) Drawdown

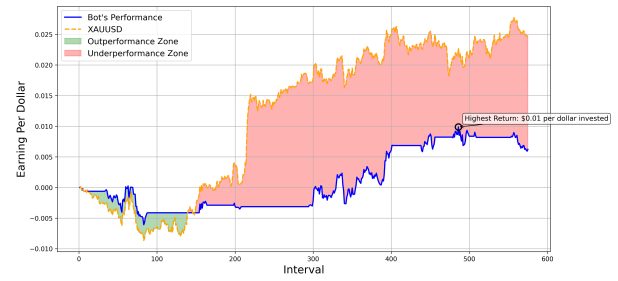


(b) Portfolio

Figure 10: Chunk 3: Drawdown and Portfolio

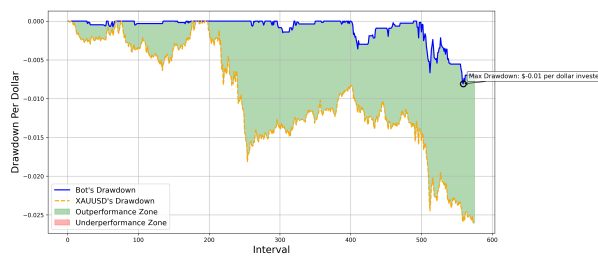


(a) Drawdown

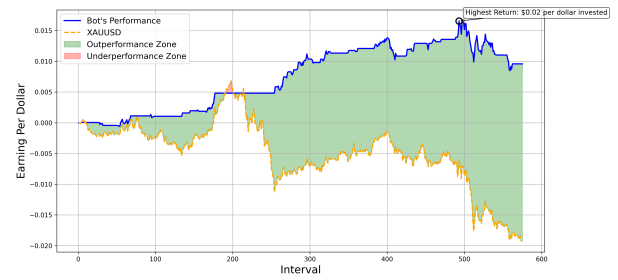


(b) Portfolio

Figure 11: Chunk 4: Drawdown and Portfolio

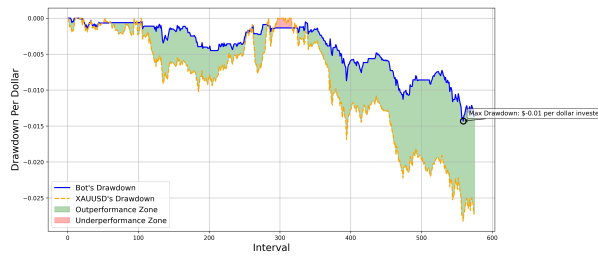


(a) Drawdown

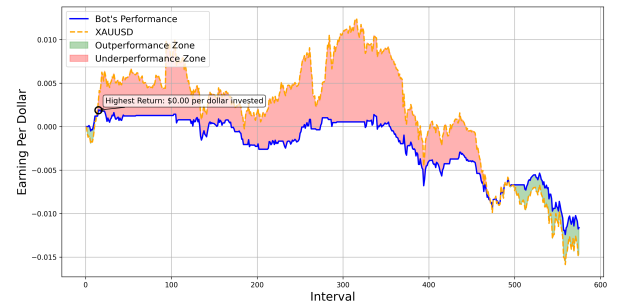


(b) Portfolio

Figure 12: Chunk 5: Drawdown and Portfolio

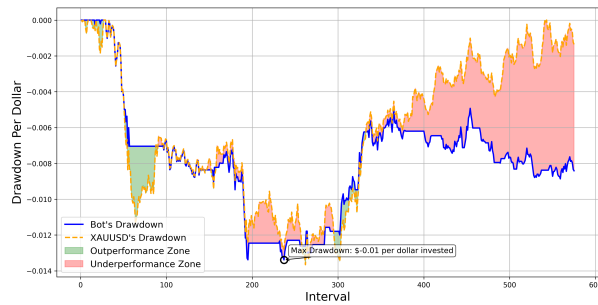


(a) Drawdown

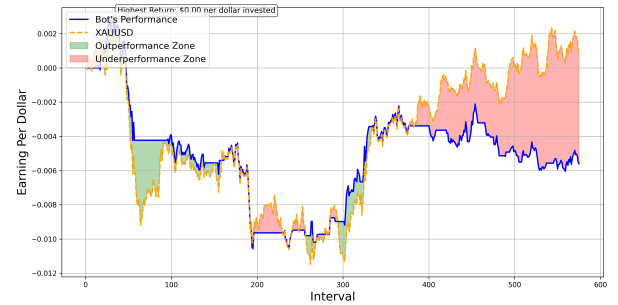


(b) Portfolio

Figure 13: Chunk 6: Drawdown and Portfolio

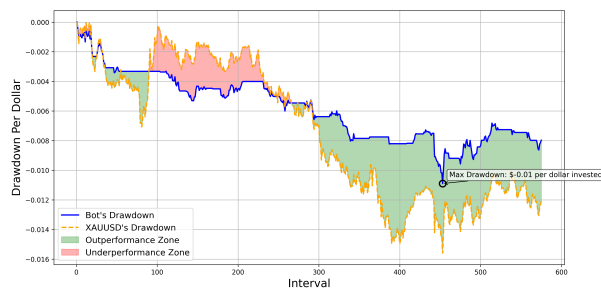


(a) Drawdown

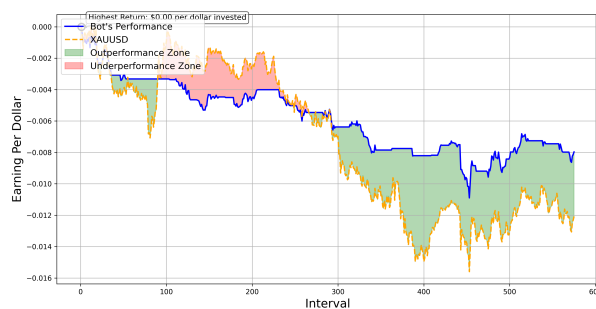


(b) Portfolio

Figure 14: Chunk 7: Drawdown and Portfolio



(a) Drawdown



(b) Portfolio

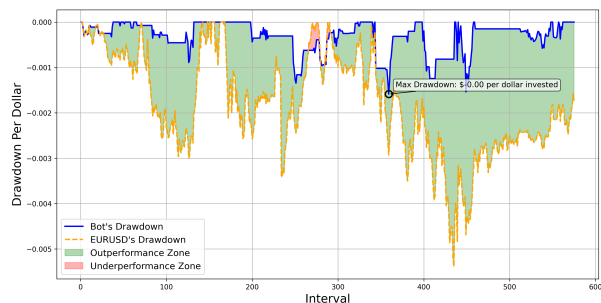
Figure 15: Chunk 8: Drawdown and Portfolio

7.4 PDT Profit vs Buy & Hold Profit (EURUSD Chunks)

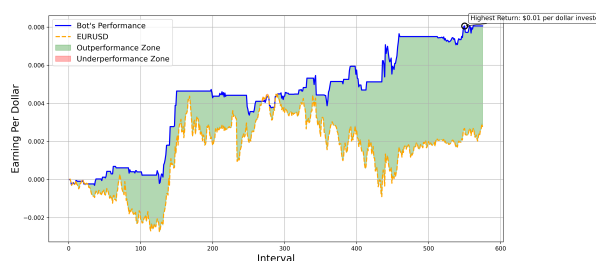
Table 8: PDT Profit vs Buy & Hold Profit for EURUSD Chunks

Chunk	PDT Profit	Buy & Hold Profit
1	0.008 06	0.002 76
2	0.000 49	-0.002 29
3	-0.001 83	-0.004 10
4	0.005 19	0.003 33
5	0.000 34	0.000 67
6	0.001 80	0.001 58
7	0.000 22	-0.005 45
8	0.007 24	0.010 32
9	0.003 63	0.010 86

7.5 Drawdown and Portfolio Charts for EURUSD Chunks

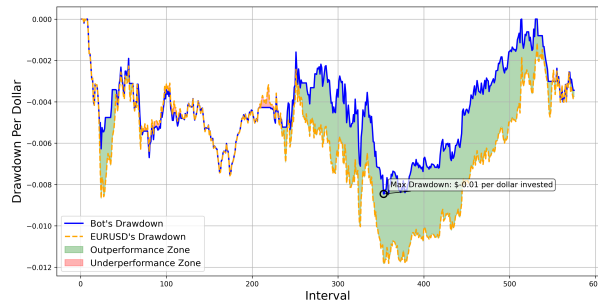


(a) Drawdown

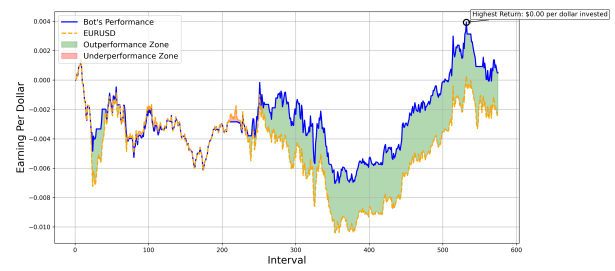


(b) Portfolio

Figure 16: Chunk 1: Drawdown and Portfolio

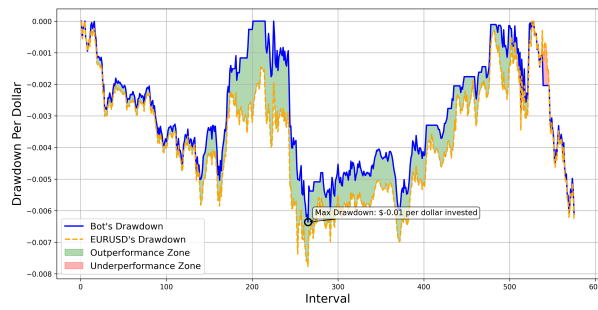


(a) Drawdown

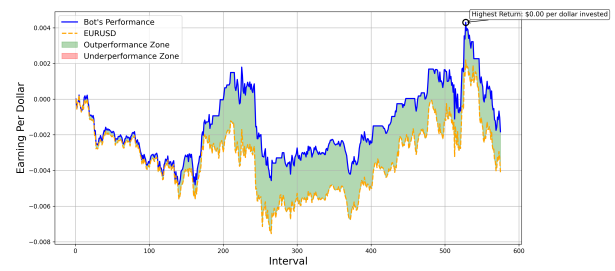


(b) Portfolio

Figure 17: Chunk 2: Drawdown and Portfolio

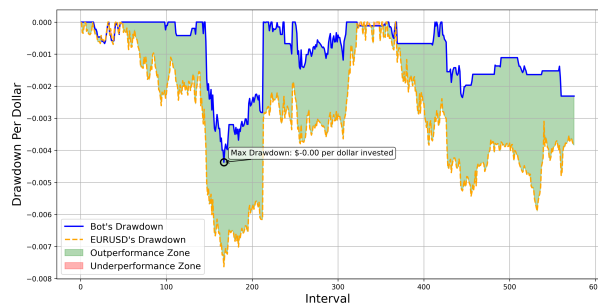


(a) Drawdown

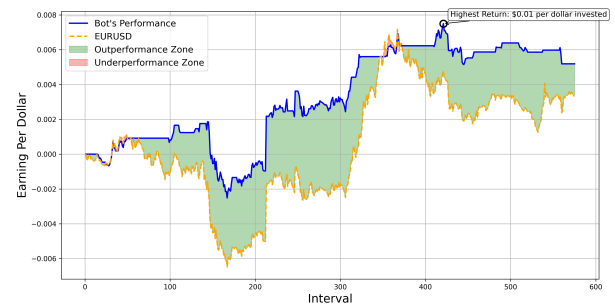


(b) Portfolio

Figure 18: Chunk 3: Drawdown and Portfolio

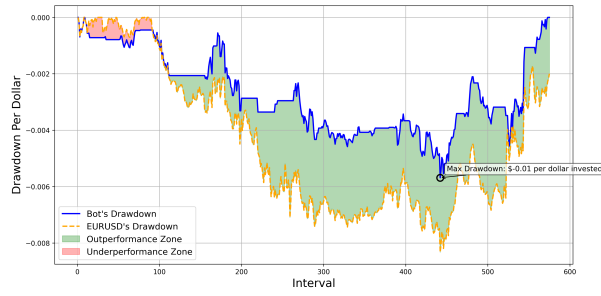


(a) Drawdown

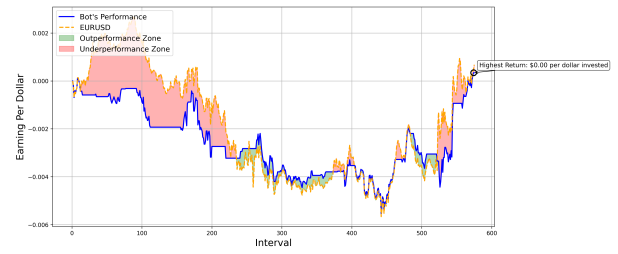


(b) Portfolio

Figure 19: Chunk 4: Drawdown and Portfolio

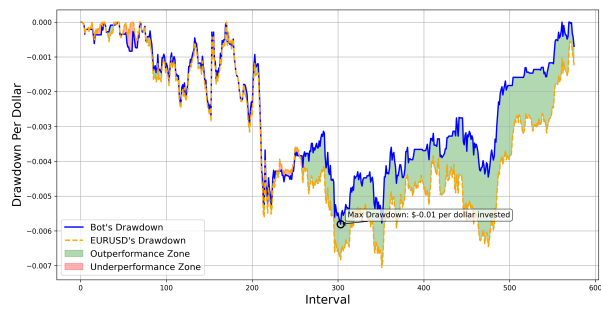


(a) Drawdown

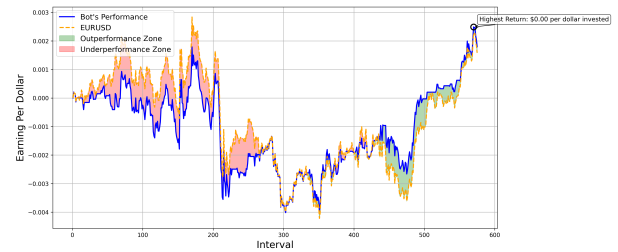


(b) Portfolio

Figure 20: Chunk 5: Drawdown and Portfolio

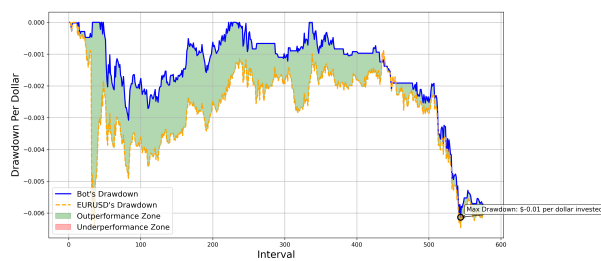


(a) Drawdown

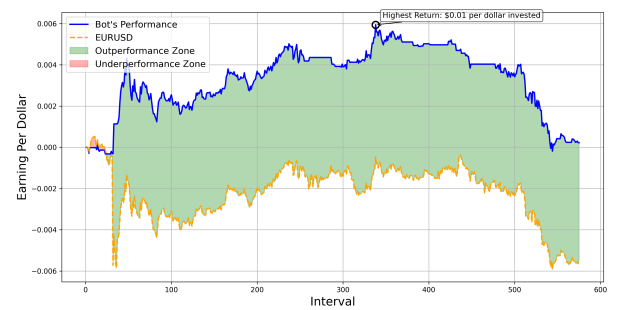


(b) Portfolio

Figure 21: Chunk 6: Drawdown and Portfolio



(a) Drawdown



(b) Portfolio

Figure 22: Chunk 7: Drawdown and Portfolio

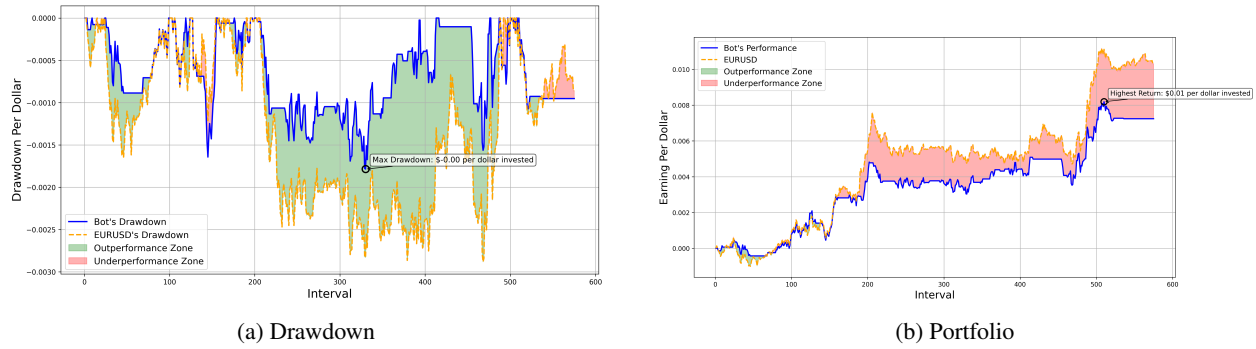


Figure 23: Chunk 8: Drawdown and Portfolio

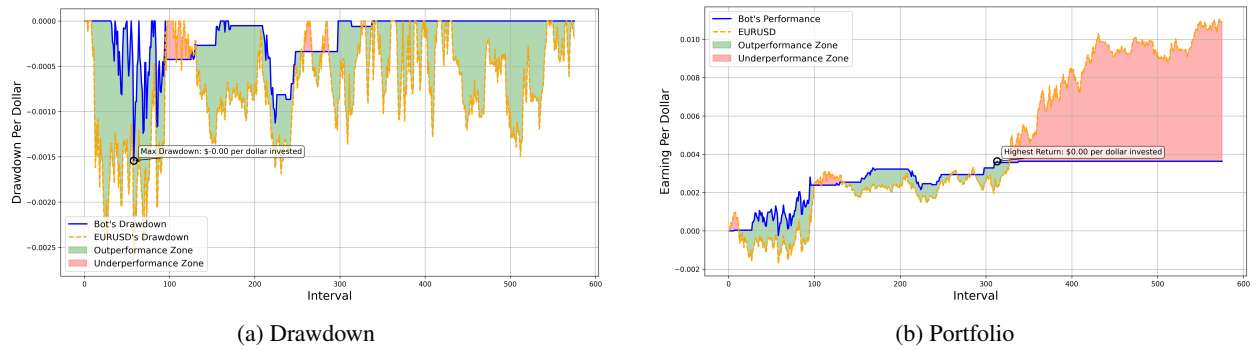


Figure 24: Chunk 9: Drawdown and Portfolio

References

- [1] Zahra Fathali, Zahra Kodia, and Lamjed Ben Said. Stock market prediction of nifty 50 index applying machine learning techniques. *Applied Artificial Intelligence*, 36(1):2111134, 2022.
- [2] Chinthakunta Manjunath, Balamurugan Marimuthu, and Bikramaditya Ghosh. Analysis of nifty 50 index stock market trends using hybrid machine learning model in quantum finance. *International Journal of Electrical and Computer Engineering (IJECE)*, 13(3):3549–3560, 2023.
- [3] Gourav Kumar and Vinod Sharma. Stock market index forecasting of nifty 50 using machine learning techniques with ann approach. *Int J Mod Comput Sci (IJMCS)*, 4(3):22–27, 2019.
- [4] R Shankar, D Sridhar, and K Sivakumar. Systematic analysis of blue-chip companies of nifty 50 index for predicting the stock market movements using anfis machine learning approach. *J. Math. Comput. Sci.*, 11(1):265–277, 2020.
- [5] Gurjeet Singh. Machine learning models in stock market prediction. *arXiv preprint arXiv:2202.09359*, 2022.
- [6] Vanshu Mahajan, Sunil Thakan, and Aashish Malik. Modeling and forecasting the volatility of nifty 50 using garch and rnn models. *Economies*, 10(5):102, 2022.
- [7] Vikas Lamba, Susheela Hooda, Vikas Solanki, Rakesh Ahuja, Sachin Ahuja, and Amandeep Kaur. Comparative analysis of artificial neural networks for predicting nifty 50 value in the indian stock market. In *2021 5th International Conference on Information Systems and Computer Networks (ISCON)*, pages 1–5. IEEE, 2021.
- [8] G Eswar Prasad, G Hemanth Kumar, B Venkata Nagesh, S Manikanth, P Kiran, et al. Prediction of financial stock market based on machine learning technique. *J Contemp Edu Theo Artific Intel: JCETAI-102*, 2023.
- [9] Jaydip Sen, Sidra Mehtab, and Abhishek Dutta. Stock price prediction using machine learning and lstm-based deep learning models. *Authorea Preprints*, 2021.
- [10] Pushpendra Singh Sisodia, Anish Gupta, Yogesh Kumar, and Gaurav Kumar Ameta. Stock market analysis and prediction for nifty50 using lstm deep learning approach. In *2022 2nd International Conference on Innovative Practices in Technology and Management (ICIPTM)*, volume 2, pages 156–161, 2022.

- [11] Syed Hasan Jafar, Shakeb Akhtar, Hani El-Chaarani, Parvez Alam Khan, and Ruaa Binsaddig. Forecasting of nifty 50 index price by using backward elimination with an lstm model. *Journal of Risk and Financial Management*, 16(10):423, 2023.
- [12] Jigar Patel, Sahil Shah, Priyank Thakkar, and Ketan Kotecha. Predicting stock and stock price index movement using trend deterministic data preparation and machine learning techniques. *Expert systems with applications*, 42(1):259–268, 2015.
- [13] Vikalp Ravi Jain, Manisha Gupta, and Raj Mohan Singh. Analysis and prediction of individual stock prices of financial sector companies in nifty50. *International Journal of Information Engineering and Electronic Business*, 13(2):33, 2018.
- [14] Harikrishnan N B, Arham Jain, and Nithin Nagaraj. Permutation decision trees. 2024.
- [15] Karthi Balasubramanian and Nithin Nagaraj. Aging and cardiovascular complexity: effect of the length of rr tachograms. *PeerJ*, 4:e2755, 2016.
- [16] N. Nagaraj, K. Balasubramanian, and S. Dey, “A new complexity measure for time series analysis and classification,” *The European Physical Journal Special Topics*, vol. 222, no. 3, pp. 847–860, 2013.
- [17] W. Ebeling and M. A. Jiménez-Montaño, “On grammars, complexity, and information measures of biological macromolecules,” *Mathematical Biosciences*, vol. 52, no. 1–2, pp. 53–71, 1980.